

兆鸣嵌入式

C语言·一个LED讲透面向对象

EP07 信息隐藏：static与头文件

三家世界500强 · 11年一线嵌入式实战

公众号搜索「兆鸣嵌入式」 回复「交流」进群

核心概念

信息隐藏的本质：把实现细节锁在.c文件里（static），只通过头文件暴露必要的接口。防的不是坏人，防的是"不小心"。

信息隐藏不是不信任——是锁上门，谁都不容易犯错

设计思路

问题：同事直接改struct成员导致bug

EP06里struct成员暴露在外，同事直接写了 `led.is_on = true`，但忘了操作GPIO——软件以为灯亮了，硬件其实没动，LED状态全乱了。这就是"状态和硬件不一致"的经典bug。

解决：static函数=后厨，头文件=菜单，getter函数安全读取

1. 头文件（.h）只暴露函数声明，就像餐厅菜单——只告诉你能点什么菜。
2. .c文件里的辅助函数用static修饰，外部根本调不到——就像后厨，顾客进不来。
3. 提供led_get_state()让外部安全读取状态，而不是直接读me->is_on。

抽出static void update_hardware()集中管理所有GPIO操作，改一处全生效——不再散落在每个函数里。

视频讲义

你的数据是敞开的

任何人都能从任何地方直接改你的 struct

你的代码 一切正常

```
/* 你的LED驱动，写得很好 */
led_t led_red;

led_init(&led_red, "PI.15", true);
led_on(&led_red); /* 正常点亮 */
led_off(&led_red); /* 正常熄灭 */

/* 一切正常，没有问题 ✓ */
```

但是
⚡
同事

同事的代码 灾难

```
/* 同事在另一个文件里写的 */
extern led_t led_red;

/* 他直接改了你的数据！ */
led_red.pin_num = 999;

/* 你的LED全不亮了 */
/* 你查了两个小时 ... */
/* 最后发现不是你的问题 */
```

struct 是你的钱 —— 你会把钱放在路边让大家自取吗？

兆鸣嵌入式

问题代码：直接改struct成员，状态与硬件不一致

第一把锁：static

加了 static，出了这个文件就不存在

就像饭店



前台

客人可以进 —— 点菜、吃饭



后厨

客人不能进 —— 做菜、备料

对应
→

你的代码

前台 —— 外面能用

```
/* 这些函数在 .h 里声明 */
void led_init(led_t *me, ...);
void led_on(led_t *me);
void led_off(led_t *me);
```

后厨 —— 加 static 锁住

```
/* 这些函数只在 .c 里 */
static void configure_gpio(...);
static void validate_pin(...);
```

static = 后厨的门锁 —— 别的文件找都找不到

兆鸣嵌入式

static函数 = 后厨，外部看不到也调不到

第二把锁：头文件 = 菜单

菜单上只写菜名，不写配方

led.h 菜单 — 别人能看到的

只放函数声明，不放实现

```
/* 菜名：别人可以点的 */

void led_init(led_t *me,
               const char *pin,
               bool level);

void led_on(led_t *me);

void led_off(led_t *me);
```

led.c 厨房 — 只有你自己知道

实现 + static 内部函数

```
/* 配方：外面看不到 */

static void configure_gpio( ... )
{
    /* 内部辅助，外面调不到 */
}

void led_on(led_t *me)
{
    configure_gpio(me->pin_num);
    platform_pin_write( ... );
}
```

分开

菜单上没有的，你就别碰 —— 这就是接口

兆鸣嵌入式

头文件 = 菜单，只暴露“能点的菜”

C语言 vs C++：信息隐藏

C 用文件当墙，C++ 用关键字当墙

C语言 你手动分

static + .h/.c 文件分离

```
/* led.h — 菜单（公开） */
void led_init( ... );
void led_on(led_t *me);
void led_off(led_t *me);

/* led.c — 后厨（私有） */
static void configure_gpio( ... );
static void validate_pin( ... );
```

C++ 编译器帮你分

private / public 关键字

```
class Led {
public: /* 菜单 */
    void init( ... );
    void on();
    void off();

private: /* 后厨 */
    void configure_gpio( ... );
    void validate_pin( ... );
};
```

=
底层相同

上期：struct + me = class + this

这期：static + .h/.c = private + public

恭喜你，C++ 面向对象的两个核心机制，你已经全懂了

兆鸣嵌入式

C vs C++ 对照：static = private

代码结构

文件	说明
led.h	菜单——函数声明 + led_get_state() 安全读取接口
led.c	后厨——static辅助函数 + 公共函数实现
main.c	演示正确用法 vs 危险用法（反面教材）
Makefile	编译脚本

关键代码解读

1. static update_hardware() —— 后厨集中管理GPIO

led.c

```
/* static = 后厨专用，外部调不到 */
static void update_hardware(Led_t *me)
{
    platform_gpio_write(me->pin, me->is_on);
}

static bool is_brightness_valid(uint8_t brightness)
{
    return (brightness <= 100);
}
```

所有GPIO操作集中在update_hardware()里。如果硬件逻辑变了（比如高低电平反转），只改这一个函数就够了。static让它只在led.c内部可见。

2. led_get_state() —— 安全的"读"接口

led.c

```
int led_get_state(const Led_t *me,
                  bool *is_on,
                  uint8_t *brightness)
{
    if (me == NULL) return -1;
    if (is_on != NULL)
        *is_on = me->is_on;
    if (brightness != NULL)
        *brightness = me->brightness;
    return 0;
}
```

通过函数读状态，养成"什么都通过接口"的好习惯。直接读me->is_on就养成了直接访问struct的习惯，迟早会手滑写成赋值。

3. 危险的反面教材 (main.c节选)

main.c (对比)

```
/* 危险！直接修改struct成员 */
red_led.is_on = true;
/* 忘了操作GPIO！硬件根本没变！ */

/* 安全！通过函数操作 */
led_on(&red_led);
/* 函数内部同时更新is_on和GPIO，不可能漏掉 */
```

C vs C++ 对照

C 写法	C++ 写法
static函数 (.c内部)	private方法
.h文件的函数声明	public方法
static update_hardware()	private updateHardware()
led_get_state()	getter方法
.h=菜单, .c=后厨	public/private分区

编译运行

```
cd code/EP07_信息隐藏
make
./demo
```

获取更多

公众号搜索「兆鸣嵌入式」

回复「交流」加群，与一线工程师切磋

抖音 / B站 / 视频号

搜索「兆鸣嵌入式」